

PROCESSOS E THREADS





Diretor Executivo

DAVID LIRA STEPHEN BARROS

Gerente Editorial

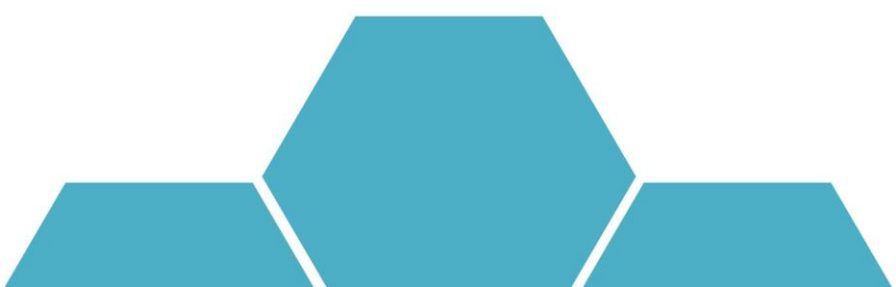
CRISTIANE SILVEIRA CESAR DE OLIVEIRA

Projeto Gráfico

TIAGO DA ROCHA

Autoria

LEANDRO C. CARDOSO



AUTORIA

Leandro C. Cardoso

Sou formado em Comunicação Social com habilitação em *Design* Digital e mestrado em Tecnologias da Inteligência e *Design* Digital, pela PUC-SP, com experiência em direção de arte e criação na área com mais de 25 anos. Passei por empresas com a Laureate International Universities - FMU | FIAM-FAAM, Universidade Anhembi Morumbi e o Centro Paula Souza (FATEC-ETEC). Fui analista de Desenvolvimento Pedagógico Sênior - Laureate EAD e coordenador de Curso Técnico de *Design* Gráfico, no Centro Paula Souza. Fui revisor técnico e validador para cursos EAD, atendendo cliente como Laureate International Universities; DeVry Brasil; UNEF; FAESF; Faculdade Positivo; Uninter; Platos Soluções Educacionais S.A. (Krotonn - Universidade Anhaguera). Sou autor de mais de 21 livros didáticos, um dos organizadores da Maratona de Criação e *Design* do Curso de Comunicação Visual, da ETEC - Albert Einstein. Sou apaixonado pelo que faço e adoro transmitir minha experiência de vida àqueles que estão iniciando em suas profissões. Por isso, fui convidado pela Editora Telesapiens a integrar seu elenco de autores independentes. Estou muito feliz em poder ajudar você nesta fase de muito estudo e trabalho. Conte comigo!

ICONOGRAFICOS

Olá. Esses ícones irão aparecer em sua trilha de aprendizagem toda vez que:



OBJETIVO:
para o início do desenvolvimento de uma nova competência;



NOTA:
quando forem necessários observações ou complementações para o seu conhecimento;



EXPLICANDO MELHOR:
algo precisa ser melhor explicado ou detalhado;



SAIBA MAIS:
textos, referências bibliográficas e links para aprofundamento do seu conhecimento;



ACESSE:
se for preciso acessar um ou mais sites para fazer download, assistir vídeos, ler textos, ouvir podcast;



ATIVIDADES:
quando alguma atividade de autoaprendizagem for aplicada;



DEFINIÇÃO:
houver necessidade de se apresentar um novo conceito;



IMPORTANTE:
as observações escritas tiveram que ser priorizadas para você;



VOCÊ SABIA?
curiosidades e indagações lúdicas sobre o tema em estudo, se forem necessárias;



REFLITA:
se houver a necessidade de chamar a atenção sobre algo a ser refletido ou discutido sobre;



RESUMINDO:
quando for preciso se fazer um resumo acumulativo das últimas abordagens;



TESTANDO:
quando o desenvolvimento de uma competência for concluído e questões forem explicadas;

SUMÁRIO

Conceito de processos e <i>threads</i>	10
Os processos e o modelo de <i>threads</i> na dinâmica de um sistema operacional	10
Comunicação e escalonamento de processos.....	17
Forma de comunicação entre processos de um sistema operacional	17
Técnica de escalonamento de processos em um sistema operacional.....	21
Gerenciamento de memória	24
Atribuições do gerenciamento de memória	24
Algoritmos de substituição de páginas.....	32
Políticas de substituição de páginas	32

UNIDADE

02

INTRODUÇÃO

Você sabia que os processos e *threads* são uma das demandas mais importantes do sistema operacional, justamente por serem relacionados à memória, seja ela física, a memória RAM ou virtual? Isso mesmo. Para o entendimento desses processos e *threads*, é importante compreendê-los e ao modelo de *threads* na dinâmica de um sistema operacional, a comunicação e o escalonamento de processos, como a forma de comunicação entre processos de um sistema operacional. Também é importante conhecer as técnicas de escalonamento de processos em um sistema operacional, logicamente, o gerenciamento de memória e as suas atribuições e o conhecimento dos algoritmos de substituição de páginas e como ocorre as políticas de substituição de páginas. Entendeu? Ao longo desta unidade letiva você vai mergulhar nesse universo!

OBJETIVOS

Olá. Seja muito bem-vindo à Unidade 2. Nosso objetivo é auxiliar você no desenvolvimento das seguintes competências profissionais até o término desta etapa de estudos:

1. Discernir sobre o papel dos processos na dinâmica de um sistema operacional.
2. Aplicar o modelo de *threads* em sistemas operacionais.
3. Compreender a forma de comunicação entre processos de um sistema operacional.
4. Entender a técnica de escalonamento de processos em um sistema operacional.

Conceito de processos e *threads*



OBJETIVO:

Ao término deste capítulo, você será capaz de entender como funcionam os processos e *threads*. Isto será fundamental para o exercício de sua profissão. As pessoas que tentaram compreender, sem a devida instrução, tiveram problemas ao trabalhar com sistemas operacionais. E então? Motivado para desenvolver esta competência? Então vamos lá. Avante!

Processo é uma entidade dinâmica e sua função é de alteração do seu estado, de acordo com o avanço de sua execução. O conceito de *threads* está relacionado em um processo com múltiplos fluxos de controle.

Os processos e o modelo de *threads* na dinâmica de um sistema operacional

Para entender o conceito de processos, entende-se que a totalidade dos arquivos executáveis, chamados diretamente, rodará nesses processos. Os executáveis, que são chamados em um processo, permitem rodar explicitamente, seja em outro ou no mesmo processo. Logicamente, quando esses procedimentos forem viáveis, de acordo com a característica do executável, geralmente se usa uma DLL.

Os arquivos executáveis são os que se caracterizam por realizar comandos, no momento que são abertos, ou seja, executado pelo usuário. São facilmente identificados pelas extensões EXE, BAT e COM.

DLL - *Dynamic Link Library* nada mais é que uma biblioteca dinâmica na qual estão contidos dados que podem ser acessados, por mais de um programa que esteja instalado em um sistema operacional. A DLL pode ser acessada da maneira simultânea, ou seja, mais de um programa acessa a mesma DLL, ao mesmo tempo. Um exemplo prático é o momento em que abrimos mais do que um programa ao mesmo tempo. Além de conceituar DLL, é importante entender que o processo tem controle do sistema operacional em duas situações. São elas:

1. Relacionado ao tempo de processador, no qual o processo tem referência à memória disponível, que pode ser solicitada por esse processo.
2. Nas permissões de acesso, relacionadas a outros recursos, que têm vínculos ao processo como um todo.

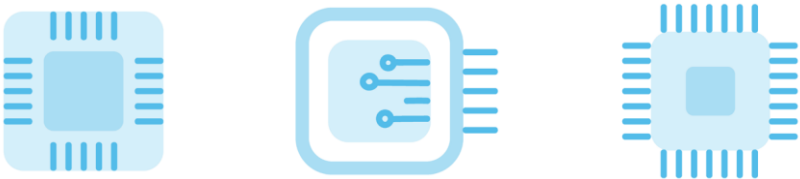
Figura 1 - Todos os arquivos executáveis rodam em um processo



Fonte: Freepik (2021).

É importante frisar que um processo tem um espaço de endereçamento de memória virtual, destinado único e, exclusivamente, para ele. De certa forma, pode ser conceituado como se o processo rodasse, unicamente, no dispositivo, como o computador.

Figura 2 - Uma das situações em que o processo tem controle do sistema operacional está relacionada ao tempo de processador

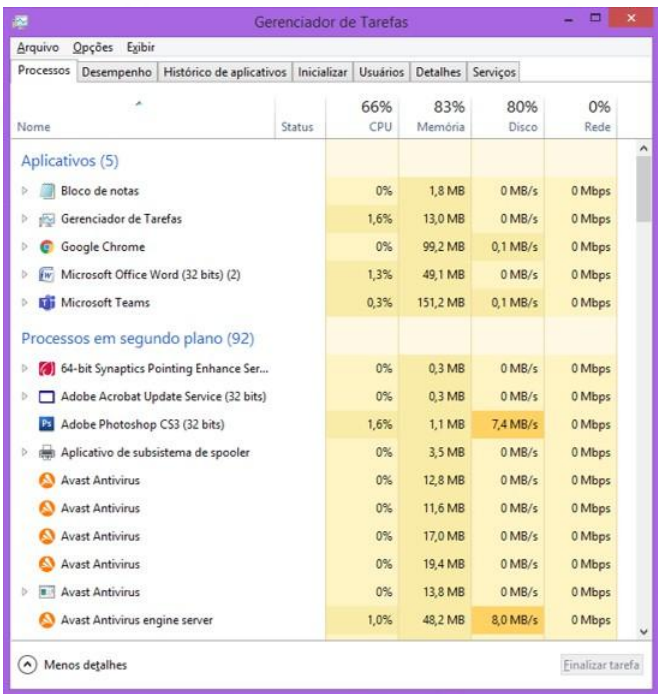


Fonte: Freepik (2021).

Embora pareça que o processo “rode sozinho” não é que o realmente acontece. Os profissionais iniciantes e estudantes, às vezes, se confundem, pelo fato que comportamento desse processo é bem característico e esteja executando de maneira única. Não acontece devido ao fato de que ele é executável, com poder de rodar em vários momentos. Estes são os conceitos, de maneira geral, relacionados a eles. Já a conceituação de *thread* pode ser definida que é similar ao de processo. Esta semelhança

ocorre pelo fato que o *thread* tem uma nova linha de execução, e, também, todos os aspectos que estão relacionados a esta linha. Assim o sistema operacional lhe dá o tratamento de maneira similar ao processo. Para alocar o tempo de processamento, mas quando se trata da memória, esta responsabilidade de aplicar o controle do acesso, é compartilhado por todo o processo. Uma maneira prática de visualizá-lo, é utilizando o sistema operacional Windows, por exemplo, por meio do Gerenciador de Tarefas, que pode ser acessado pela tecla de atalho CTRL+ALT+DEL, conforme exemplificada na próxima figura.

Figura 3 - No gerenciador de tarefas, do Sistema Operacional Windows, é uma das formas de visualizar os processos



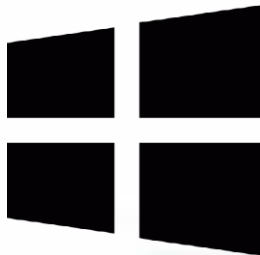
Fonte: Elaborado pelo autor (2021).

Para entender melhor em relação aos processos, é importante salientar que, comumente, os programas que também são conhecidos como aplicações têm uma *stack* para cada *thread*. Mas raramente um programa tem mais do que um *stack* espaço de *heap* para todo o sistema. Dessa maneira, alguns profissionais encontram dificuldades na

programação com *threads*, pelo fato de ser muito difícil compartilhar estado. A definição de *stack*, também conhecida como pilha de funções, trata-se de uma área da memória em que são alocadas as variáveis ou ponteiros. No momento, em que uma função é chamada e desalocada, como também no momento que esta função é encerrada, em outras palavras seria a representação da memória local àquela função.

Já o termo *heap*, também conhecido como área de alocação dinâmica, é um espaço reservado para variáveis e dados, criado no período da execução do programa, também conhecido como expressão *runtime*. Em outras palavras, o *heap* pode ser considerado como a memória global do programa ou aplicação, assim, *threads* estão vinculados ao processo, até porque o principal em si não deixa de ser uma *thread*. É importante considerar que alguns programas dispõem de seu próprio controle de *threads* internamente, não controlados pelo processador ou o sistema operacional.

Figura 4 - Algumas aplicações dispõem de seu próprio controle de *threads* internamente não controladas pelo sistema operacional - Windows.



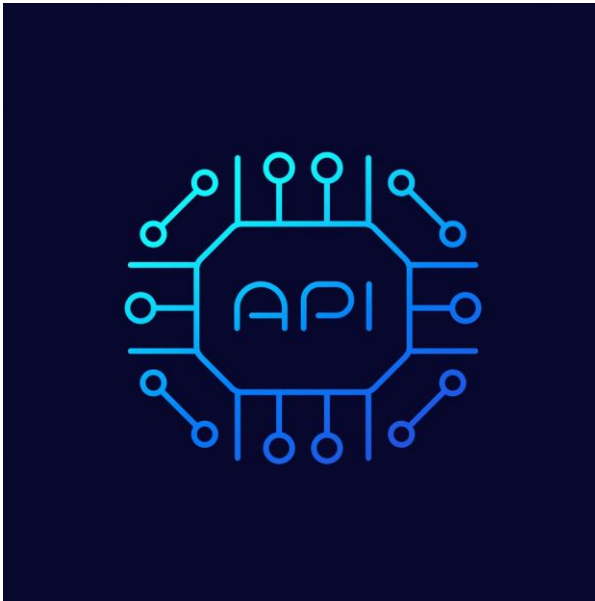
Fonte: Freepik (2021).

As vantagens, quando os *threads* não são controlados pelo processador/ sistema operacional, é que pode ser evitada a troca de contexto do sistema operacional, mas também tem desvantagens, por exemplo, não ser permitido o uso de outros processadores. Geralmente, são denominados como *soft*, *light* ou *green threads*, para entender melhor, *green threads* são implementados no nível do programa, e não no sistema operacional. Normalmente, acontecem no momento que o sistema operacional não fornece uma API de *thread*. Dessa maneira, os benefícios são obter as funcionalidades similares a do *thread* propriamente dito.

**IMPORTANTE:**

API - *Application Programming Interface* (Interface de Programação de Aplicações) trata-se de uma série tanto de padrões como de rotinas, estabelecidas por uma aplicação, com objetivo de usar de suas funções por aplicativos, nos quais não pretendem envolver-se em detalhes da implementação do programa, mas somente utilizar suas funcionalidades.

Figura 5 - API- conjunto de padrões e rotinas estabelecidas por um programa



Fonte: Freepik (2021).

De maneira geral, de acordo com a maneira que é analisada, *thread* pode ser visto correspondente a um processo ou não, mas analisando, especificamente, na perspectiva de agendamento do processador. Nesta visão, pode-se conceituar que apenas existem *threads*, e não processos, e de maneira simplificada, na visão de gerenciamento de memória e acesso aos recursos externos, pode-se definir que só existe o processo. Veja a seguir uma tabela comparativa entre processo e *threads*.

Quadro 1- Comparação entre processo e *threads*

Recurso	Processo	Thread
Execução	Linha própria	Linha própria
Memória global	Própria	Compartilhada
Memória local	Sim	Geralmente
Consumo de memória	Normal	Ligeiramente menor
Manipuladores de recursos externos	Proprietário	Empresta do processo
Tempo de criação	Relativamente longo	Relativamente curto
Tempo de troca de contexto	Relativamente longo	Relativamente curto
Instâncias	Múltiplas	Múltiplas
Associação	Um programa (executável)	Um processo
Paralelismo	Limitado	Sim
Comunicação entre seus pares	Só com um mecanismo de IPC	Sim, no processo
Eficiência de comunicação	Não	Sim
Comunicação direta com o OS	Sim	Não
Controle de exceções	Próprio	Próprio
Confiabilidade e segurança	Sim	Depende do código

Fonte: Silberschatz (2010).

De maneira geral, processo trata de uma entidade dinâmica, cuja função é de alteração de seu estado, conforme avança sua execução. Já *threads* são um processo com múltiplos fluxos de controle.



RESUMINDO:

E então? Gostou do que lhe mostramos? Aprendeu mesmo tudinho? Agora, só para termos certeza de que você realmente entendeu o tema de estudo deste capítulo, vamos resumir tudo o que vimos. Você deve ter aprendido que os conceitos de processos e *threads*, embora tenham similaridades são diferentes, pois processos são referentes a uma entidade dinâmica, que tem com atribuição modificar seu estado, de acordo com o avanço de sua execução. Em relação à conceituação de *threads* é um processo com múltiplos fluxos de controle e processos são todos praticamente arquivos de programas executáveis de maneira geral. Os programas dispõem de uma *stack* para cada *thread*, quase nunca um programa tem mais do que um *stack*, espaço de *heap* para todo processo. *Stack*, nada mais é do que pilha de funções, a área da memória na qual são alocados variáveis ou ponteiros. Existem vantagens e desvantagens no momento que os *threads* não são controlados pelo processador e ou sistema operacional. Uma delas é evitar a troca de contexto do sistema operacional, mas não permite o uso de outros processadores.

Comunicação e escalonamento de processos



OBJETIVO:

Ao término deste capítulo, você será capaz de entender o processo de comunicação e escalonamento em um sistema operacional. Isto será fundamental para o exercício de sua profissão. E então? Motivado para desenvolver esta competência? Então vamos lá. Avante!

A sigla IPC, da língua inglesa, *Inter-Process Communication*, ou simplesmente comunicação entre processos, consiste em um grupo de mecanismos que viabiliza aos processos o poder de transferir as informações entre si.

Forma de comunicação entre processos de um sistema operacional

Executar um processo presume, em relação do sistema operacional, entre vários aspectos, criar um contexto de execução próprio do qual, certa maneira, é abstraído o processo dos componentes reais do sistema. No sistema operacional Windows, por exemplo, ele pode ser visualizado no gerenciador de tarefas, no qual são discriminados os aplicativos, esses processos, em segundo plano, e os próprios processos do sistema operacional.

Figura 6 - No Windows, por exemplo, os processos podem ser visualizados no gerenciador de tarefas

Gerenciador de Tarefas					
Arquivo Opções Exibir					
Processos Desempenho Histórico de aplicativos Inicializar Usuários Detalhes Serviços					
Nome	Status	22% CPU	81% Memória	17% Disco	0% Rede
Aplicativos (6)					
▸ Bloco de notas		0%	2,4 MB	0 MB/s	0 Mbps
▸ Gerenciador de Tarefas		5,8%	12,3 MB	0 MB/s	0 Mbps
▸ Google Chrome		2,6%	78,4 MB	0 MB/s	0 Mbps
▸ Microsoft Office Word (32 bits) (2)		0,1%	51,1 MB	0 MB/s	0 Mbps

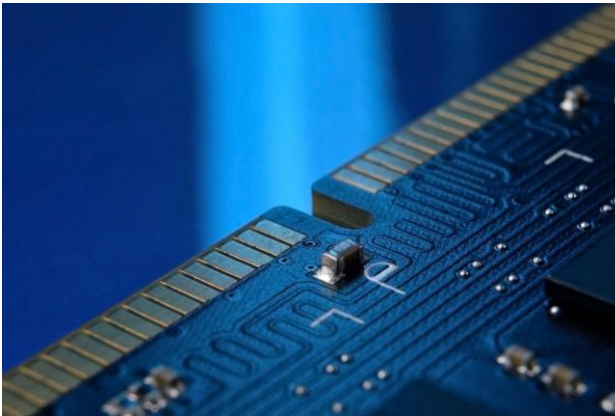
Fonte: Elaborado pelo autor(2021).

Justamente pelo fato da virtualização dos recursos, faz com que o próprio processo não tome ciência em relação aos outros e, desta maneira, não consiga intercambiar informações.

Técnica que viabiliza que um programa de um sistema operacional, ou até mesmo o sistema na sua totalidade, seja executado em outro sistema operacional, é o conceito de virtualização. Por exemplo, permite que o sistema Linux seja executado no Windows e vice-versa.

Os modelos de comunicação entre processos, normalmente, são caracterizados em dois grupos. O primeiro recebe o nome de memória compartilhada e o segundo de troca de mensagens. O modelo de memória compartilhada caracteriza-se por ser uma área de memória na qual, como o nome diz, é compartilhada entre os processos. Já segundo modelo acontece por meio da troca dessas mensagens entre os processos.

Figura 7 - No modelo de memória compartilhada, há o compartilhar entre os processos



Fonte: Freepik (2021).

O objetivo principal da memória compartilhada, além de permitir o acesso simultâneo de vários programas, é promover a comunicação entre eles, evitando que aconteçam cópias redundantes. As cópias redundantes consistem em duplicar os componentes principais, ou seja, os mais críticos, com objetivo de aumentar a confiança de um sistema e sua disponibilidade. De maneira geral, a memória compartilhada, de acordo com a situação, permite que os programas sejam executados, apenas em um processador, ou também por dois processadores diferentes. Deve-se

atentar que nos fundamentos de memória compartilhada, normalmente, não é inclusa a utilização da memória com objetivo de comunicar distintos *threads* de um mesmo processo.

Figura 8- A memória compartilhada viabiliza que as aplicações sejam executadas apenas em um ou dois processadores diferentes



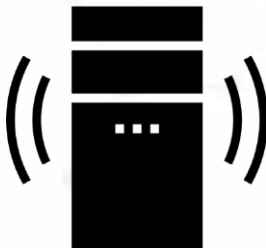
Fonte: Freepik (2021).

Para compreender melhor a comunicação de processos, também, é importante o entendimento da sincronização entre processos, no qual tem a função de proporcionar o gerenciamento do acesso concorrente a recursos do sistema operacional, mas de maneira que se tenha controle em relação aos processos, proporcionando que um recurso não sofra alterações simultâneas, ou que os processos não fiquem aguardando que o recurso seja disponibilizado. Os mecanismos de sincronização podem ser classificados quanto à espera em dois grupos:

1. *Busy waiting* - espera ociosa.
2. Espera bloqueante.

Em relação à espera ociosa, a entidade seja processo ou *thread*, tem o seu funcionamento testando de maneira repetitiva à condição de sincronização. Normalmente é usado uma variável de impedimento que é denominada *spinlock*, que é um bloqueio que viabiliza com que um *thread* que procura adquiri-lo, apenas aguarde um *loop* até que seja verificado de forma repetida, se o bloqueio está com disponibilidade.

Figura 9- A memória compartilhada apresenta como desvantagem o desperdício de tempo de CPU



Fonte: Freepik (2021).

A memória compartilhada apresenta como desvantagem, se a espera é longa, o desperdício de tempo de CPU. Assim, normalmente, é usada nos programas que executam processamento paralelo, conhecido como multiprocessamento, e recomendada para sincronização rápida.

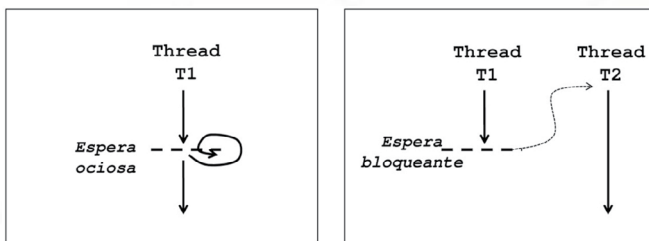


DEFINIÇÃO:

Central Process Unit - CPU, ou simplesmente em português, Unidade Central de Processamento, é considerado o principal elemento de *hardware* do computador, cuja responsabilidade é desenvolver cálculos e a realização de específicas tarefas, de acordo solicitada pelo usuário.

A espera bloqueante é a segunda classificação dos mecanismos de sincronização. Neste caso não existe desperdício do tempo de CPU, no momento que é executado um programa no modo usuário. Uma comparação ilustrativa entre a espera ociosa e a espera bloqueante pode ser visualizada na Figura 10.

Figura 10 - Comparação entre a espera ociosa e a espera bloqueante



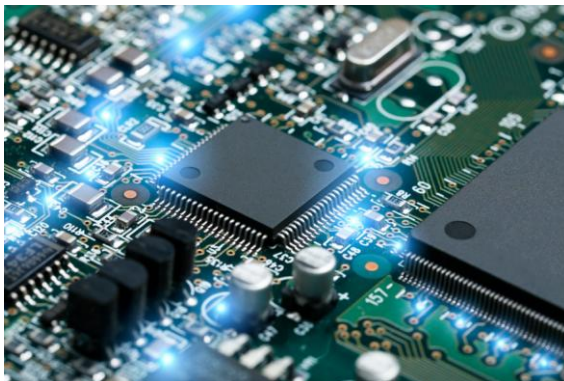
Fonte: Elaborado pelo autor (2021).

Comparando à espera ociosa e a espera bloqueante, fica nítido que no primeiro método, ou seja, a espera ociosa ou *loop*, e no segundo método de espera bloqueante, não há o desperdício de tempo de CPU.

Técnica de escalonamento de processos em um sistema operacional

Escalonamento de maneira geral é o ato de decisão no qual o processo irá ocupar o processador no momento que ele estiver livre, e para facilitar essa etapa, geralmente, os sistemas operacionais dispõem de um escalonador, ou seja, o responsável pelas decisões para resultar em uma escolha, no qual geralmente se baseia na política, usando os algoritmos de escalonamento, atribuindo assim os processos concorrentes a um ou mais processadores. Normalmente, o escalonamento é executado pelo sistema operacional, com base no compartilhamento do processador, por meio de técnicas de *timesharing*, no qual o tempo de utilização do processador é dividido em *time-slice*, ou seja, pequenas fatias de tempo que são usadas pelos variados processos, ordenadamente, permitindo assim, o uso de forma eficiente o processador que está disponível.

Figura 11 - O uso de maneira eficiente do processador é função do escalonador do sistema operacional



Fonte: Freepik (2021).

Para que seja garantida a imparcialidade, o sistema operacional, normalmente, define a ativação ou suspensão dos processos, por meio de uma técnica chamada de preempção, ou seja, de preferência.

Preempção ou preemptividade, nos sistemas operacionais, é a ação de interrupção temporária de uma tarefa que está em execução, por meio de um sistema operacional, mas com objetivo de retornar a tarefa posteriormente; esse ato também é conhecido como troca de contexto.

Escalarar pode apresentar problemas como apontam Casavant e Kuhl (1988), visualizando como recurso para o gerenciamento de recursos. Assim é apresentado o problema dividindo em três componentes principais:

1. Algoritmo de escalonamento.
2. Consumidores que são os processos tanto dos sistemas quanto dos usuários.
3. Recursos que podem ser redes de comunicação, memória processadores, entre outros.

De maneira geral, os consumidores, ou seja, os processos dos usuários e dos sistemas operacionais usam os recursos computacionais como também o algoritmo de escalonamento, que é o responsável pela a distribuição dos processos entre os processadores, conforme se faz necessário, e também são disponíveis nos recursos.

Figura 12 - As redes de comunicação são uma das situações de aplicação do escalamento para a definição de prioridades



Autores como Shirazi e Hurson (1992) e Baumgartner e Wah (1991) apontam que distribuir processos é importante que seja vinculado com um propósito, que deve ter como base todas as decisões de escalonamento a serem tomadas. Para exemplificar, uma decisão pode ser tomada com o intuito da diminuição do tempo de execução dos processos, balanceando, assim a carga do sistema.



RESUMINDO:

E então? Gostou do que lhe mostramos? Aprendeu mesmo tudo? Agora, só para termos certeza de que você realmente entendeu o tema de estudo deste capítulo, vamos resumir tudo o que vimos. Você deve ter aprendido que a comunicação e o escalonamento de processos são muito importantes para um bom funcionamento de um sistema operacional, que fica evidenciado pelo uso da sigla IPC *Inter-Process Communication*, que significa comunicação entre processos, relacionados a uma série de mecanismos que viabiliza aos processos o poder de transferir as informações entre si. A execução de um processo considera o relacionamento do sistema operacional em diferentes aspectos, como desenvolver um contexto de execução própria que é abstraída do processo dos componentes reais do sistema. De modo geral, os modelos de comunicação entre processos, são divididos em duas classificações grupos, o de memória compartilhada e troca de mensagens, este último tem como base a troca de mensagens entre os processos. Já a memória compartilhada trata-se de uma área de memória no qual é compartilhada entre os processos.

Gerenciamento de memória



OBJETIVO:

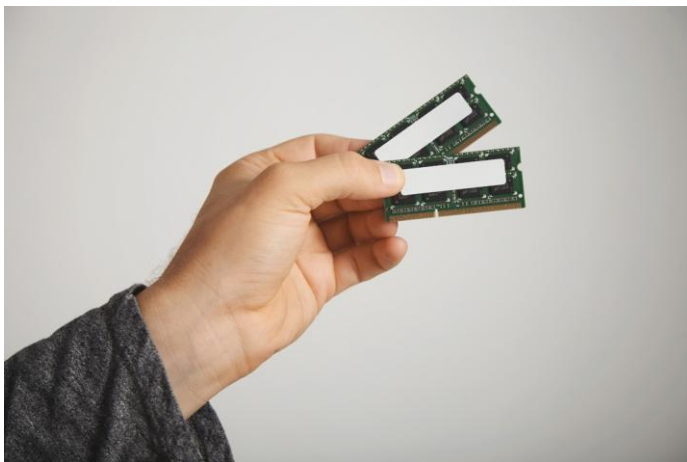
Ao término deste capítulo, você será capaz de compreender como ocorre o gerenciamento de memória dos sistemas operacionais. Isto será fundamental para o exercício de sua profissão. E então? Motivado para desenvolver esta competência? Então vamos lá. Avante!

O gerenciamento da memória RAM que está disponível em um dispositivo como um computador, é considerado uma das principais características e atribuições do sistema operacional, para que ele possa assegurar que o próprio sistema como também quaisquer processos tenham a quantidade de memória necessária, para a execução dos processos.

Atribuições do gerenciamento de memória

Quando comparamos aos demais recursos disponíveis em um sistema operacional, como a instalação de programas, entre outros, podemos considerar que a memória dispõe de recursos escassos.

Figura 13- O gerenciamento da memória RAM e as atribuições do sistema operacional



Pelo fato de que os recursos da memória são considerados escassos, várias técnicas foram criadas com o objetivo de otimizar o seu uso. É importante estarmos atualizados e procurar informações detalhadas de cada uma, principalmente nas atualizações dos sistemas operacionais. São exemplos, segundo Vahalia (1996):

- Compressão de memória.
- Bibliotecas dinâmicas compartilhadas.
- Memória virtual.
- Carga de páginas sob demanda.
- *Copy-on-write*.
- Alocação preguiçosa.

Quando um sistema é virtualizado, sua característica permite que em cada máquina virtual se obtenha do hipervisor uma fração da memória da máquina, no qual tem o gerenciamento do sistema operacional convidado. Para entender melhor o hipervisor, trata-se de um programa que desenvolve e executa máquinas virtuais, também conhecido como VMs ou VMM - monitor da máquina virtual.

Figura 14 - Em um sistema virtualizado, cada máquina virtual obtém do hipervisor uma fração da memória da máquina

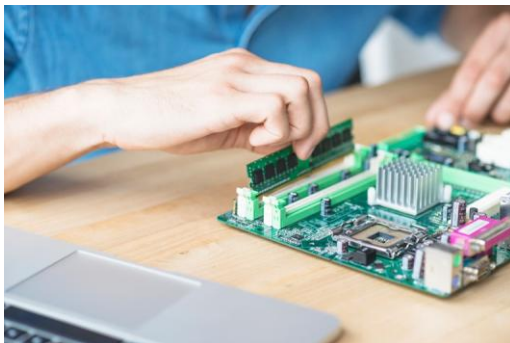


É importante salientar que os mecanismos que estão, internamente, no sistema operacional têm limitação em relação à memória da própria máquina virtual, na qual tem isolamento das demais pelo hipervisor. Para facilitar o entendimento em relação ao gerenciamento de memória nos sistemas virtualizados, alguns termos devem ser facilmente compreendidos, segundo VMware Inc. (2010):

- Memória física da máquina virtual - trata-se da memória que é disponibilizada pelo hipervisor, gerenciada pelo sistema operacional da máquina virtual - VM.
- Memória virtual da máquina virtual - está relacionado à memória vista pelos processos da máquina virtual, ou seja, as páginas.
- Memória de máquina - *host memory* - trata-se da memória RAM, propriamente dita, ou seja, a memória real que está disponível no *hardware* e também gerenciada pelo hipervisor.

Mecanismos relacionados à alocação de memória, geralmente devem ser adicionados, nos sistemas virtualizados, com o objetivo de prestar atendimentos à demanda de memória dos sistemas convidados.

Figura 15- A memória de máquina - *host memory*- tem a memória RAM propriamente dita

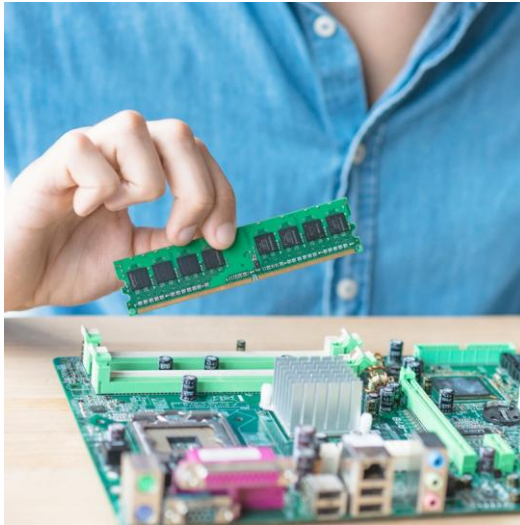


Fonte: Freepik (2021).

Ao serem adicionados mecanismos facilitam tanto na manutenção quanto no equilíbrio de recursos nos ambientes, principalmente, em ambientes de grande porte com diversas máquinas virtuais - VM, consequentemente, ocasionando aproveitamento do *hardware* e economia de recursos físicos.

O recurso denominado *memory overcommit*, também, se refere na proposta de oferecimento de espaço de memória, além da memória física que está disponível no dispositivo como um computador, mas sem quaisquer garantias de que a quantidade do armazenamento físico, determinada efetivamente exista, desta maneira, é um recurso também usado nos sistemas operacionais. É importante considerar que geralmente os sistemas convidados utilizam somente pouca parte da memória física que lhe são alocadas, permitindo que os hipervisores com a função de identificação de trechos de memória ócios, atuem bem. De modo dinâmico, executam o processo de relocação para as outras máquinas virtuais - VMs, que precisam de quantidade de memória maior no momento específico, segundo VMware Inc. (2010).

Figura 16- O *memory overcommit* oferece espaço de memória e memória física que está disponível



Fonte: Freepik (2021).

Além do recurso *memory overcommit*, existem outras técnicas de recuperação e de otimização de memória, por exemplo:

- *Swap*.
- *Memory compression*.
- *Ballooning*.

A técnica denominada de *swap*, geralmente, é utilizada quando se esgotam todas as opções de recuperar páginas de memória por meio de outras técnicas como *ballooning* e *memory compression*.



DEFINIÇÃO:

Página virtual, página de memória, ou simplesmente uma página, trata-se de um bloco contínuo e de comprimento fixo de memória virtual, o qual tem descrição por uma única entrada na tabela de páginas.

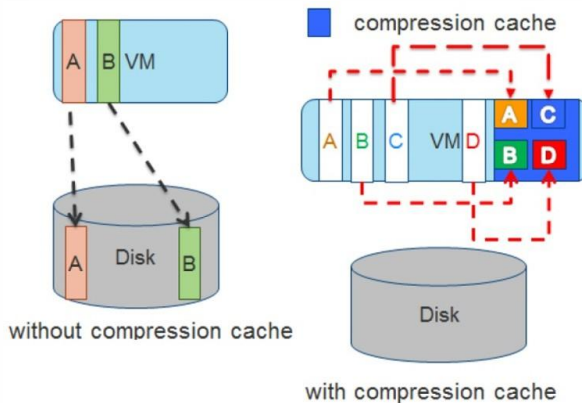
O *swap* também é utilizado quando não existe a possibilidade de alocamento de quantidade de memória maior para as máquinas virtuais - VMs, ou para o *host* de virtualização, sendo a utilização da área de troca como o último recurso para recuperação de memória (YUN *et al.*, 2014).

No *swap*, cada um dos hipervisores usa mecanismos para potencializar o processo. Geralmente, é desenvolvido um arquivo auxiliar na estrutura de diretórios do hipervisor, e nesse arquivo são transferidas as páginas de memória da máquina virtual - VM. Desta maneira, é liberado o espaço da área de memória que no momento está em disco. Importante salientar que quando acontece esse processo, não acontece interação com o sistema convidado, sendo o hipervisor o responsável por decidir quais páginas que serão enviadas para *swap* (WALDSPURGER, 2002). É considerado desvantagem que reside no hipervisor o fato de não dispor o acesso às páginas, as quais seriam mais recomendadas de serem remetida para *swap*. Assim, uma máquina virtual - VM pode acessar uma página na mesa hora que é enviada para área de intercâmbio, podendo afetar de forma considerável a *performance* do sistema convidado.

A técnica de recuperação e de otimização de memória que recebe o nome de *memory compression*, os mecanismos de compressão de memória são usados em situações de falta de memória do hipervisor. E como quaisquer sistemas, na inexistência de memória, as páginas serão direcionadas para a área de troca ou *swap*, isso se não existir a possibilidade de recuperação da memória a suficiente, usando a técnica denominada como *balloning*.

Ou outras técnicas de gerenciamento de memória, em que as páginas que iriam para área de troca podem sofrer compressão e serem armazenadas em uma unidade na qual há o controle do hipervisor, que recebe o nome de cache de compressão (WALDSPURGER, 2002). Esta ficará situada na memória do *host*, para que possam ser direcionadas para o cache de compressão as páginas, que foram identificadas para a área de troca que precisam de percentual de ao menos 50% de compressão. Na situação que esse valor não seja atingindo, essa mesma página não é preiteada ao cache de compressão, segundo VMware Inc. (2010). A figura 18 representa o processo de armazenamento de páginas e o cache de compressão. Observe que com a utilização do cache de compressão não é usada a área de troca.

Figura 18 - Técnica Memory Compression

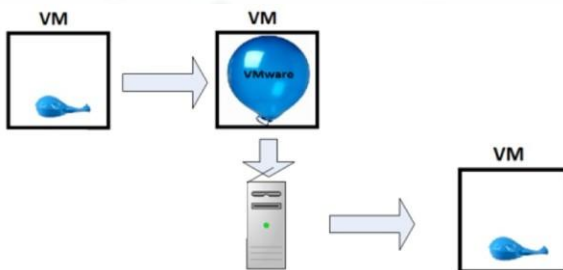


Fonte: VMware Inc (2010).

Já a técnica denominada como *ballooning* é comum em diversos hipervisores e se caracteriza por uma maneira de máquina virtual - VM, se comunicar com o hipervisor no momento que o hipervisor está demandando mais memória. De acordo com Wen *et al.*, (2012) por causa do isolamento, o sistema operacional convidado não tem ciência de que está sendo executado internamente de uma máquina virtual - VM, e, também, não tem conhecimento da relação dos estados das outras máquinas virtuais no mesmo *host* físico.

A próxima figura representa o processo de funcionamento do mecanismo *ballooning*. Observe que é somente ativado no momento que *host* está com pouca memória. E o *driver ballooning* específico em que as páginas de memória -as máquinas virtuais - VM, de certa forma serão permitidas de “abrir mão” para que o servidor executar *swap* para o disco.

Figura 19- Técnica *Ballooning*



Fonte: VMware Inc. (2010).

No momento que o hipervisor executa diversas máquinas virtuais - VMs- e o número total de memória livre no *host* está baixo, nenhuma das máquinas virtuais poderão efetuar a liberação física, pois o sistema operacional convidado não tem ciência da falta de memória do *host*. Em relação à abordagem desse tema, entre outros, é importante estar atualizado, principalmente, pelo fato que sempre há novas atualizações dos sistemas operacionais.



RESUMINDO:

E então? Gostou do que lhe mostramos? Aprendeu mesmo? Agora, só para termos certeza de que você realmente entendeu o tema de estudo deste capítulo, vamos resumir tudo o que vimos. Você deve ter aprendido que o gerenciamento da memória, seja ele físico como a memória RAM que está disponível em um dispositivo, como um computador, ou memória virtual, é considerado uma das principais características e atribuições do sistema operacional. Para isso, é importante o conhecimento de virtualização, entendendo suas aplicações, tipos e técnicas, um diferencial no entendimento do gerenciamento de memória em ambientes virtualizados. Importante salientar que apesar de existirem diversos mecanismos de recuperação de memória, os ambientes virtualizados precisam de uma quantidade de memória adicional para controle do funcionamento do hipervisor. Enfim, o gerenciamento de memória é importante para que o próprio sistema operacional e quaisquer processos tenham a quantidade de memória necessária.

Algoritmos de substituição de páginas



OBJETIVO:

Ao término deste capítulo, você será capaz de compreender o que são os algoritmos de substituição de páginas e as suas funções no sistema operacional. Isto será fundamental para o exercício de sua profissão. E então? Motivado para desenvolver esta competência? Então vamos lá. Avante!

No momento da gerência da memória virtual com paginação, um dos maiores problemas que, geralmente, são apresentados, por incrível que pareça, não estão relacionados à decisão de qual página deve ser carregada na memória principal. Mas sim, qual será a página que deve ser removida da memória principal, justamente os algoritmos de substituição de página têm como principal atribuição de seleção dos *frames* que terão menor chances de serem referenciados em um futuro próximo.

Políticas de substituição de páginas

A política de substituição de páginas usadas é considerada um dos elementos mais fundamentais em um sistema de memória virtual, pois tem poder de determinar os parâmetros de seleção que serão utilizados para remoção de uma ou mais páginas de memória principal, no momento que se faz necessário que outra página seja designada e não exista espaço disponível.

Para implementar a substituição de página, usam-se algoritmos para tal substituição que de maneira geral são classificados em dois tipos:

1. Algoritmos com espaço variável.
2. Algoritmos com espaço fixo.

Figura 20 - Algoritmos - decisão da substituição de uma página



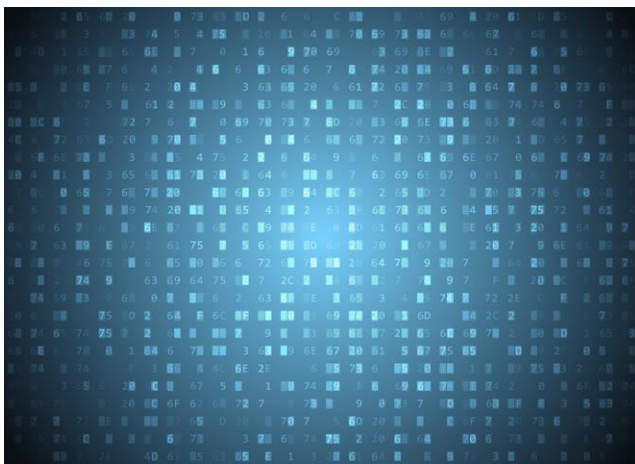
Fonte: VMware Inc. (2010).

Os algoritmos com espaço variável caracterizam-se pelo poder de alterar o tamanho da memória, alocada dinamicamente, conforme o volume de execução do sistema. Já os algoritmos com espaço fixo caracterizam-se em relação a uma determinada região de memória que esteja disponível de maneira contínua, ou seja, sempre constante. São exemplos de algoritmos com espaço fixo:

- FIFO.
- LRU.
- OPT.
- MRU-n.
- Clock.
- NRU.
- LFU.
- MFU.
- WS.
- PFF.
- VMIN.

O algoritmo FIFO - *First In, First Out*- tem, como diretriz de substituição, remover da memória a página carregada que está há mais tempo, não considerando acessos, identificados no momento que esteve residente na memória.

Figura 21 - O algoritmo FIFO remove da memória virtual a página carregada há mais tempo



Fonte: Freepik (2021).

Alguns estudiosos apontam que o algoritmo FIFO pode apresentar uma incapacidade que recebe o nome de “anomalia de Belady” (BELADY; NELSON; SHEDLER, 1969), que se caracteriza pelo número de faltas de página, ocasionalmente, gerar um aumento no momento que o tamanho da memória disponível aumenta. Mas tem como benefício a facilidade de implementar e simplicidade, pois uma fila pode ser facilmente usada como ordenamento das páginas, de acordo com a ordem de chegada, por exemplo. Uma página como regra entrará no começo da fila no momento que é carregada e a página substituída, como diretriz, sempre será aquela que ocupará a última posição da fila.

Figura 22 - Uma página entrará no começo da fila no momento que é carregada



Fonte: Freepik (2021).

O algoritmo *Least Recently Used* - LRU - é outro exemplo de algoritmo de espaço físico. É considerado bastante eficiente e com simplicidade, utilizando como diretriz de substituição a página que teve acesso há mais tempo. Desta forma, será a página residente que esteja com menor atividade ativa no momento da substituição; é importante estar sempre atualizados, pois este algoritmo tem algumas variações, nos quais podem ser destacadas duas:

1. LRU-K.
2. 2Q.

O algoritmo denominado como LRU-K, segundo O'Neil *et.al* (1993, 1999), não considera a quantidade de tempo que uma página foi acessada pela última vez, mas sim o momento que aconteceu o seu último acesso. Para exemplificar, veja a seguinte situação: o LRU-2 tem a função de verificar qual a página que realizou penúltimo acesso há mais tempo, neste mesmo momento o LRU-3 identifica o antepenúltimo acesso e assim de maneira sucessiva.

Em relação ao algoritmo denominado como 2Q, como apontam Johnson e Shasha (1994), tem como diretriz utilizar duas áreas para gerenciar a memória que são:

1. Páginas recém-carregadas.
2. Páginas acessadas no momento que estavam na primeira área.

As páginas que são excluídas da memória, utilizando as diretrizes 2Q são as que, geralmente, estão na primeira área, e a definição utilizada são os critérios FIFO, mas caso, a situação for que a primeira área se torne pequena demais, e se quase todas as páginas que foram carregadas já foram acessadas pelo menos por duas vezes. O critério de substituição executado será em relação a uma página da segunda área, a diretriz aplicada nesta situação será o LRU. Para entender melhor, nesta situação a memória é dividida em dois níveis, pois as páginas serão direcionadas em cada um, conforme sua frequência de utilização. Assim, pode-se observar que o algoritmo 2Q usa uma convenção bem considerável dos algoritmos LRU e FIFO para critérios de substituição das páginas.

O algoritmo que recebe o nome OPT ou Ótimo, de acordo com Belady, (1966), tem como função representar a melhor solução possível, com objetivo de minimizar o número de faltas de página.

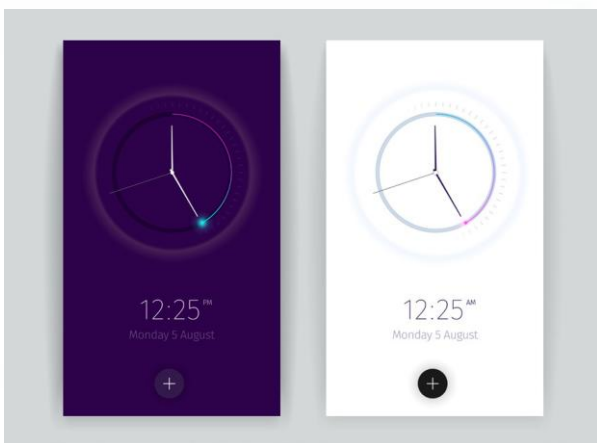


IMPORTANTE:

O algoritmo OPT utiliza como critério remover da memória, em caso de falta, a página que leva maior tempo para ser novamente referenciada, pode se dizer que o algoritmo “conhece” o futuro, pelo fato de permitir a leitura prévia de todos os acessos à memória, que foi executada pelo programa, o que não aconteceria em uma situação real de execução (BELADY, 1966).

Já o algoritmo que recebe o nome de MRU-n de *Most Recently Used*-tem como diretriz escolher a última página que foi acessada, para que ela seja substituída, é uma variação do MRU-2. Mas é considerado mais eficiente, pois explora, com maior ou menor grau de intensidade, as características de localidade temporal que algumas páginas apresentam. Já o *clock*, se caracteriza por ser uma fila como a do FIFO, no qual é usada pelo algoritmo do relógio, ou algoritmo *clock*, em que as páginas são apresentadas nesta fila, conforme a ordenação que foram carregadas na memória.

Figura 23 - *Clock* tem como característica de ser uma fila, usada pelo algoritmo do relógio



Fonte: Freepik (2021).

Neste algoritmo, a página substituída geralmente a que foi carregada há mais tempo, mas neste caso um bit de uso será adicionado ao mecanismo de seleção, fazendo com que a página residente mais antiga apenas seja substituída. Na situação no qual o seu bit de uso estiver zerado, se esta situação não ocorrer, a próxima página com bit zerado na sequência da fila será selecionada.



DEFINIÇÃO:

O bit é considerado a menor unidade de informação que pode ser tanto transmitida quanto armazenada na informática. Os algoritmos bit são considerados bastante velozes e, geralmente, são utilizados para melhorar a *performance* de programas.

A função do bit no *clock* é de prestar informação da situação de uma página se ela foi ou não referenciada, desde o momento do seu carregamento, utilizando este procedimento não acontece o risco de remoção de páginas que foram acessadas recente, assim é similar ao LRU.

Figura 24 - Um bit de uso é adicionado ao mecanismo de seleção, na página substituída



Fonte: Freepik (2021).

O algoritmo NRU - *Not Recently Used* - tem como diretriz a remoção da memória de uma página não acessada, recentemente, isto é, entre as duas últimas faltas. Este algoritmo também tem a função de verificação de cada página residente foi ou não alterada, isto é, se aconteceram modificações nos dados, desde que foi carregada na memória. Neste caso, dois bits são utilizados:

1. Primeiro com objetivo de representação da existência ou não de acessos recentes.
2. Segundo com objetivo de sinalizar, se a página foi modificada, desde o seu carregamento.

A diretriz que estabelece prioridades de substituição utiliza a ordem, de primeiro as páginas não referenciadas e também não alteradas, segundo as páginas apenas não referenciadas, terceiro, as páginas que não foram alteradas e, quarto, as páginas referenciadas e também modificadas.

Figura 25 - No algoritmo NRU, caso aconteçam modificações nos dados de uma página, dois bits são utilizados

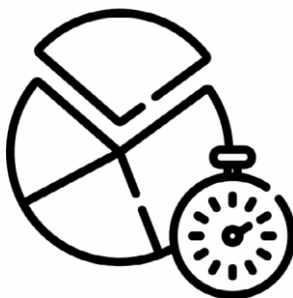


Fonte: Freepik(2021).

No caso do algoritmo LFU - *Least Frequently Used* - é utilizado como diretriz de substituição, a página que foi menos acessada entre as páginas que estejam carregadas na memória. Assim, esta verificação é executada por contador de acessos, também chamado de *hits*, mas associado a cada página residente, no qual é zerado no momento que uma página deixa a memória principal. Desta forma, frequência de utilização se refere somente aos acessos recentes e não a todo o histórico de execução da aplicação.

O algoritmo MFU - *Most Frequently Used* - tem como principal função substituir sempre a página residente que obteve mais acesso, igualmente, o LFU, utiliza contadores de acesso para definir a frequência de utilização de cada página. O algoritmo WS *Working Set*, segundo Denning e Schwartz, (1972), utiliza as mesmas diretrizes da LRU, no momento que a memória já está carregada e uma falta de página acontece, a página acessada há mais tempo é substituída. Porém, não tem como função somente a substituição de páginas, mas também de estabelecer o tempo máximo que cada uma pode permanecer na memória, sem ser referenciada.

Figura 27- Algoritmo WS Working Set estabelece o tempo máximo que cada página pode permanecer na memória



Fonte: Freepik(2021).

O algoritmo PFF- *Page Fault Frequency*- usa as diretrizes LRU, em situações de substituição, no momento no qual o espaço de memória disponível está totalmente ocupado, igualmente ao WS remove, automaticamente, páginas da memória, quando constatado inatividade aparente. O algoritmo VMIN é considerado bastante eficiente por reconhecer os acessos futuros que um programa irá realizar, removendo memória das páginas que o próximo acesso leve mais tempo para acontecer do que um parâmetro fixo pré-estabelecido. Por esse motivo a memória que tem o gerenciamento de VMIN, quase que nunca estará totalmente ocupada, pelo fato de apenas permanecer aquelas páginas que certamente serão referenciadas em um futuro breve.



RESUMINDO:

E então? Gostou do que lhe mostramos? Aprendeu mesmo tudinho? Agora, só para termos certeza de que você realmente entendeu o tema de estudo deste capítulo, vamos resumir tudo o que vimos. Você deve ter aprendido que os algoritmos de substituição de páginas são fundamentais para uma boa *performance* de um sistema operacional, no qual está relacionado ao gerenciamento de memória virtual, ligada à paginação. A principal decisão não é a definição de qual página será carregada na memória principal e sim, qual será a página excluída da memória principal. Para essa decisão importante são consultados os algoritmos de substituição. Assim a sua principal função é selecionar os *frames* que terão menor chances de serem referenciados em um futuro próximo, para isso, é estabelecida uma política de substituição de páginas usada, assim considerada, uns dos elementos essenciais em um sistema de memória virtual. Os algoritmos de substituição de páginas, de maneira geral, são classificados em dois tipos: algoritmos com espaço variável e algoritmos com espaço fixo.

REFERÊNCIAS

BELADY, L. A. *A study of replacement algorithms for a virtual storage computer. IBM Systems Journal*, v. 5, n. 2, p. 78-101, 1966.

BAUMGARTNER, K.M.; WAH, B.W. *A global load balancing strategy for a distributed computer system. In: International workshop on the future trends in distributed computing systems*. 1990. S, 1, 1998, Hong Kong. *Anais...* Los Alamitos: IEEE CS Press, 1998. p. 93-102.

BELADY, L.A.; NELSON, R.A.; SHEDLER, G.S. *An anomaly in space-time characteristics of certain programs running in a paging machine. Communications of the ACM*, v. 12, n. 6, p. 349-353, 1969.

CASAVANT, L.; KUHL, J.G. *A taxonomy of scheduling in general-purpose distributed computing systems*. New York: IEEE Trans. Softw. Eng, 1988.

DENNING, P.J.; SCHWARTZ, S.C. *Properties of the working-set model. Communications of the ACM*, v. 15, n. 3, p. 191-198, 1972.

VAHALIA, V. U. *UNIX Internals: The New Frontiers*. [s.l.]: Prentice-Hall, 1996.

